

apuntesnumpy.pdf



LucusSales



Cálculo y análisis numérico



1º Grado en Inteligencia Artificial



Facultad de Informática
Universidad de A Coruña



[Accede al documento original](#)

INTERCAMBIA y AHORRA en LA VUELTA A LA UNI

5 AÑOS DE GARANTÍA en todo

webuy.com



"□" → texto (cadena de caracteres).

NumPy

Types (tipos de formato)

'str' → cadena de caracteres (texto)

'int' → números enteros

'float' → números reales

Para imprimir → `print(□)`

Para verdadero o falso → `isinstance(□, □)`
 ↑ la variable
 ↓ el tipo que queremos verificar

`π = np.pi`

`coseno = np.cos`

`seno = np.sin`

`tangente = np.tan`

`arccoseno = np.arcsin`

`arccoseno = np.arccos`

`arccotangente = np.arctan`

`e□ = np.exp(□)`

`ln(□) = np.log(□)`

`2□ = np.exp2(□)`

`log2 □ = np.log2(□)`

`log10 □ = np.log10(□)`

`sinh(□) = np.sinh(□)`

`cosh(□) = np.cosh(□)`

`tanh(□) = np.tanh(□)`

`arcsinh(□) = np.arcsinh(□)`

`arcosh(□) = np.arccosh(□)`

`artanh(□) = np.arctanh(□)`

LISTAS

Ejemplo:

`lista = [2, -3.0, "hola", [1, 2]]`

para seleccionar valores se hace así:

`lista[0]`

`2`

`lista[0], lista[3]`

`[2, [1, 2]]`

En Python los valores comienzan en 0.



Escanéame

WUJAH

Scanned with
CS CamScanner

Array $\neq$ Listas

El array permite hacer operaciones.

El array tiene que tener todos los elementos del mismo tipo, la lista no.

Se escribe:

```
array = np.array([[]])
```

Para crear un vector de 0 se usa: `np.zeros(cantidad de 0□)`

Por defecto los números de los vectores en NumPy es `float64`.

Ejemplo:

```
u = np.zeros(10)
```

`print(u.dtype)` → dtype se utiliza para comprobar el tipo de número

`float64`

[Si queremos utilizar otro tipo de número distinto a `float64` se usa dtype].

Ejemplo:

```
v = np.zeros(5, dtype = int)
```

```
print(v.dtype)
```

`int32`

Si sumas dos vectores y uno de ellos es de tipo 'int' y el otro 'float', el 'int' pasa a ser 'float' para realizar la operación.

• Para conocer la longitud de un vector tenemos la función: `len`

Ejemplo:

```
u = np.zeros(10)
```

```
print(len(u))
```

`10`

No se pueden operar dos vectores de distinto tamaño.

- Para modificar las componentes de un vector se hace desde sus índices.

Ejemplo:

```
u = np.zeros(10)
```

```
print(u)
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

```
u[0] = 10.0
```

```
u[3] = -4.3
```

```
u[9] = 1.0
```

```
print(u)
```

```
[10.  0.  0. -4.3  0.  0.  0.  0.  0.  1.]
```

• Para comprobar la dimensión de un vector se usa: `shape`

Ejemplo:

```
u = np.zeros(10)
```

```
print(u.shape)
```

```
(10,)
```

En los vectores solo hay una dirección. *

* Pero en el caso de las matrices indica las dimensiones en cada una de las direcciones.

Ejemplo:

```
A = np.zeros((2,3), dtype=int)
```

```
print(A)
```

```
print(A.shape)
```

```
[[0,0,0]
 [0,0,0]]
(2,3)
```

Usamos números enteros porque si lo dejamos en `float` toma la dimensión del vector como un número decimal y da error.

Creamos así una matriz. El primer n° son las filas y el segundo n° las columnas.



- Para crear un vector de 1 se utiliza: `np.ones`

Ejemplo:

```
w = np.ones(5)
```

```
print(w)
```

```
[1. 1. 1. 1. 1.]
```

- Para crear un vector con valores aleatorios se usa: `np.random.rand`

Ejemplo:

```
w = np.random.rand(3)
```

```
print(w)
```

```
[0.7822383 0.74737891 0.72815283]
```

- Para crear un vector a partir de una lista se usa: `np.array`

Ejemplo:

```
u = [4.0, 8.0, 9.0, 11.0] # lista
```

```
w = np.array(u) # array
```

```
print(w)
```

```
[4., 8., 9., 11.]
```

- Para crear un vector con valores enteros consecutivos usamos: `np.arange`

Ejemplo:

```
u = np.arange(6)
```

```
print(u)
```

```
print(u.dtype)
```

```
[0, 1, 2, 3, 4, 5]  
int32
```

→ n° de elementos

El 6 no está incluido porque el rango de valores comienza en el 0.



Escanéame

WUOLAH

Scanned with
CS CamScanner

- Para cambiar el número en el que empieza se hace así:

Ejemplo:

```
u = np.arange(2,6)
```

```
print(u)
```

```
[2,3,4,5]
```

• Otra opción para crear un vector es: `np.linspace`

Esta función crea un vector de números equiespaciados entre el primer y el segundo valor dado, generando tantos números como sea indicado en el tercer valor.

- A diferencia de la función 'arange' estos números son del tipo `float64` (arange son `int32`).

Ejemplo:

```
w = np.linspace(0, 10, 6)
```

↗ número en el que acaba

↗ cantidad de números que presenta

```
print(w)
```

↗ número en el que comienza

```
print(w.dtype)
```

```
[0., 2., 4., 6., 8., 10.]
```

```
float64
```

Unas prácticas para acabar. Un primer trabajo para aprender. Todo eso empieza aquí.

Ingeniería, ADE, Marketing... Si quieres empezar tu carrera en el área comercial, este es tu sitio.



Descubre cómo

repsol

- Los vectores en NumPy soportan operaciones aritméticas básicas, como el producto por un escalar, sumas y restas.

Ejemplo:

```
a = np.array([1.0, 0.2, 1.2])
```

```
b = np.array([2.0, 0.1, 2.1])
```

```
print(a)
```

```
print(b)
```

```
c = 10 * a # Producto de todos los elementos de a por un escalar
```

```
print(c)
```

```
c = a + b
```

```
print(c)
```

```
[1.0, 0.2, 1.2]
[2.0, 0.1, 2.1]
[10., 2., 12.]
[3., 0.3, 3.3]
```

- Para elevar las componentes de un vector podemos:

```
a = np.array([2, 3, 4])
```

```
print(a**2)
```

```
[4, 9, 16]
```

```
a = np.array([2, 3, 4])
```

```
b = a**2
```

```
print(b)
```

```
[4, 9, 16]
```



También se pueden aplicar las funciones de cálculo usual a cada una de las componentes de un vector con una sola instrucción.

Creamos un vector

```
a = np.array([0.0, np.pi/2, np.pi, 3*np.pi/2])
```

```
c = np.linspace(0., 3*np.pi/2, 4)
```

'a' y 'c' son el mismo vector, pero obtenidos de distinta forma

```
b = np.sin(a)
```

```
print(b) # También podríamos usar 'c' (sería lo mismo)
```

```
[0.0, 1.0, 1.2246468e-16, -1.0]
```

```
c = np.zeros(len(a))
```

```
for i in range(len(a)):
```

```
    c[i] = np.sin(a[i])
```

```
print(c)
```

```
[0.0, 1.0, 1.2246468e-16, -1.0]
```

→ Otra forma de hacerlo, pero es menos eficiente.



Rebanado de vectores

Es muy habitual tener que extraer un subconjunto de un vector dado para crear un nuevo vector.

Se denomina *troceado* o *rebanado* de vectores (*array slicing*).

Se utiliza para ello ' $\square:\square$ '

Ejemplo:

```
a = np.random.rand(5)
```

```
print(a)
```

```
(0.834, 0.351, 0.392, 0.601, 0.260)
```

```
b = a[:] # Desde el principio hasta el final
```

```
print(b)
```

```
(0.834, 0.351, 0.392, 0.601, 0.260)
```

```
b = a[1:3] # Desde el índice 1 hasta el 3 (sin coger el 3)
```

```
print(b)
```

```
(0.351, 0.392)
```

```
b = a[2:-1] # Desde el índice 2 hasta el -1 (para los índices negativos empezamos  
contado desde el final).  
# sin cogerlo
```

```
print(b)
```

```
(0.392, 0.601)
```

```
b = a[:3] # Desde el principio hasta el índice 3 (sin cogerlo).
```

```
print(b)
```

```
(0.834, 0.351, 0.392)
```

```
b = a[4:] # Desde el índice 4 hasta el final.
```

```
print(b)
```

```
(0.260)
```

A diferencia de contar por la izquierda, cuando el índice es negativo contamos por la derecha, siendo el -1 el último elemento.

-Para matrices:

Ejemplo:

```
a = np.array([1.3, 0], [0, 2.0])
```

```
print(a)
```

```
[1.3, 0.]  
[0., 2.]
```

```
b = a[0, :] # Extraemos la 1ª fila
```

```
print(b)
```

```
[1.3, 0.]
```

```
b = a[1, :] # Extraemos la segunda fila
```

```
print(b)
```

```
[0., 2.]
```

```
b = a[0, :1] # Extraemos la 1ª columna de la 1ª fila
```

```
print(b)
```

```
[1.3]
```

Para extraer columnas:

```
b = a[:, 0]
```

```
print(b)
```

```
[1.3 0.]
```

```
b = a[:, 1]
```

```
[0. 2.]
```

Representación de funciones

Para poder representar funciones reales de una variable real debemos importar matplotlib.pyplot

```
import matplotlib.pyplot as plt
```



Para representar una función primero debemos definir esa función.

Ejemplo:

def f1(x):

$y = 0,2x^2 - \pi x - 5$ # Así definimos $f_1(x) = 0,2x^2 - \pi x - 5$

return y

Escribimos el vector sobre el que queremos 'x'.

$x = \text{np.linspace}(0., 50 * \text{np.pi}, 50)$

Podemos añadir etiquetas a la gráfica:

- plt.xlabel (Info. eje x)
- plt.ylabel (Info. eje y)
- plt.title (Título)
- plt.legend (Leyenda)

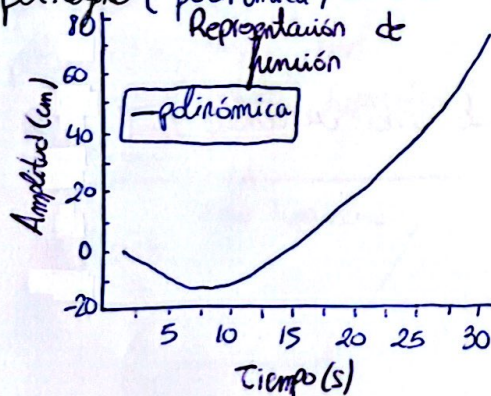
plt.xlabel("Tiempo(s)")

plt.ylabel("Amplitud(cm)")

plt.title("Representación de función")

plt.plot(x, f1(x)) → comando para representar

plt.legend("polinómica") → la leyenda después del 'plot'



También podemos crear una gráfica con: lambda

Ejemplo:

$$a = -5$$

$$b = 5$$

$$x = \text{np.linspace}(a, b, 50)$$

$$fsh = \text{lambda } x: \text{np.sinh}(x)$$

$$ysinh = fsh(x)$$

`plt.figure()` → para crear una nueva figura

`plt.plot(x, ysinh, label="sinh")`

`plt.title("Seno hiperbólico")`

`plt.legend()`

$$fch = \text{lambda } x: \text{np.cosh}(x)$$

$$ycosh = fch(x)$$

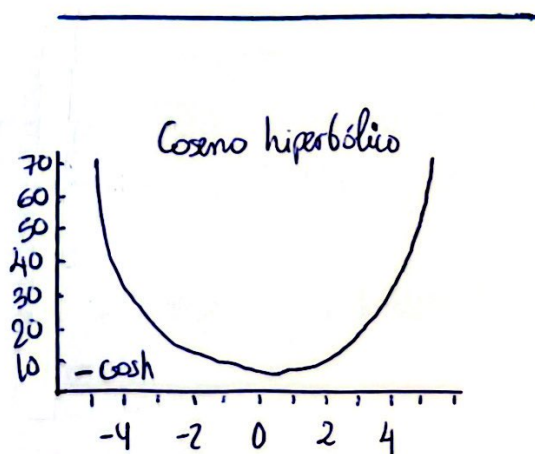
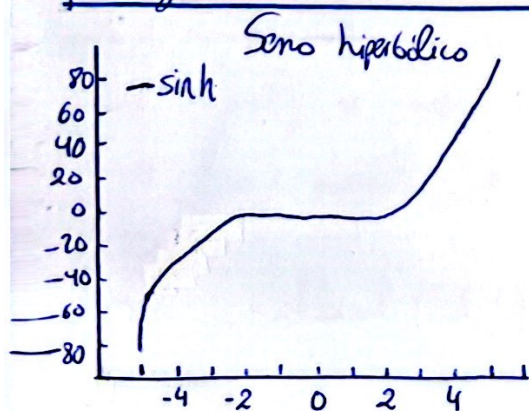
`plt.figure()`

`plt.plot(x, ycosh, label="cosh")`

`plt.title("Coseno hiperbólico")`

`plt.legend()`

Aquí la leyenda se pone en `plt.plot` y después ponemos `plt.legend`



- Función definida a trozos: `np.piecewise`

Ejemplo:

$$f(x) = \begin{cases} -x, & x \in [-2, 0] \\ \sqrt{x}, & x \in [0, 6] \end{cases}$$

$$\text{np.sqrt} = \sqrt{}$$

`x = np.linspace(-2, 6, 50)` → Aquí definimos el intervalo

`y_trozos = np.piecewise(x, [x < 0, x >= 0], [lambda x: -x, lambda x: np.sqrt(x)])`

`plt.figure()`

`plt.plot(x, y_trozos, label='y_trozos')`

`plt.title('Función a trozos')`

`plt.legend()`

→ En esto indicamos en que pto. cambia

→ y aquí indicamos la función para el intervalo respectivo

